Unit-7: Software Maintenance

Software Evolution:

- Process of developing software initially, and then timely updating it for various reasons
- Process varies based on the type of software, development methods, and people involved.
- **Software changes are inevitable due to these factors:**
  - New requirements emerge when the software is used;
  - The business environment changes;
  - Errors must be repaired;
  - The performance or reliability of the system may have to be improved, etc.
- **Software Evolution Process:**
  - Receiving change requests for new features or fixes.
  - Analyzing the impact of the change on the existing system.
  - Planning the change (deciding on **Fault Repair, Platform Adaptation** or **System Enhancement**) for a future release.
  - Implementing the change through coding and testing.
  - Releasing the updated system to users.

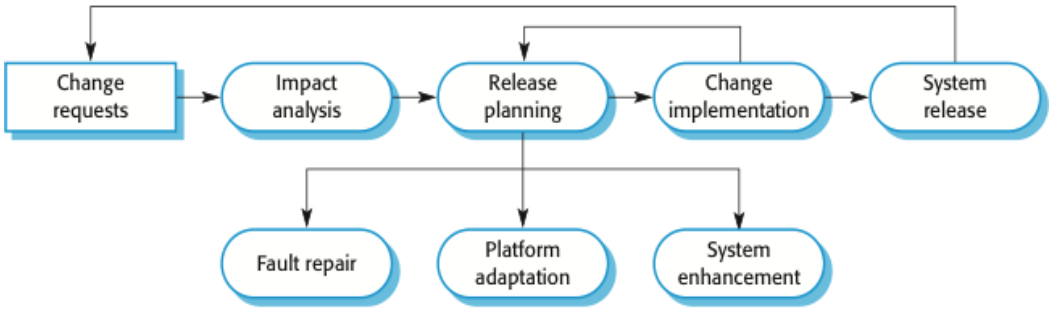


Fig: The Software evolution process

Program Evolution Dynamics:

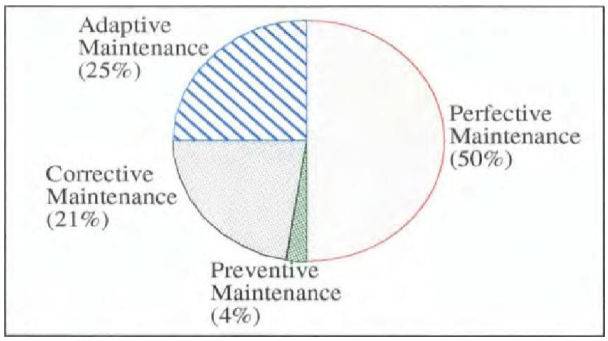
- Study of how software systems change over time.
- Lehman and Belady pioneered this field in the 1970s-80s by studying the evolution of large software systems.
- From their studies, they formulated Lehman's Laws—a set of invariant rules that describe how all large software systems evolve.
- The different proposed laws along with their description are as follows:

Law	Description
Continuing change	**Software must keep changing** to adapt to its real-world environment. **If it doesn't, it becomes less useful over time.**
Increasing complexity	**As software gets updated, its internal structure becomes messier and more complex.** You **need to spend extra effort to clean it up and keep it simple.**
Large program evolution	The **process of updating big software is stable.** The **size of changes, time between updates, and number of bugs tend to be similar for each new version.**
Organizational stability	Over a program's lifetime, its **rate of development is approximately constant and independent of the resources devoted to system development.**
Conservation of familiarity	The **amount of change in each new update is usually about the same. Developers avoid making huge, disruptive changes in a single release.**
Continuing growth	**To keep users happy, new features and functionality must be added** to the system on a regular basis.
Declining quality	The **software's quality will slowly worsen unless you actively work to maintain and update it** for new technologies and needs.
Feedback system	**Evolution is driven by feedback from users, developers, and the market.** To improve the product, you must listen to and manage this feedback.

Software Maintenance:

- Process of **modifying and updating software after delivery** to correct faults, improve performance, or adapt it to a new environment.
- **Why it is Done?**
  - To **fix errors/bugs found after release.**
  - To **improve performance or efficiency.**
  - To **adapt software to new hardware, OS, or business needs.**
  - To **add new features** requested by users.

- **4 major activities occur within maintenance:**
  - Obtaining maintenance requests
  - Transforming requests into changes
  - Designing changes
  - Implementing changes



- **Types of Software Maintenance:**
  - **Corrective maintenance:**
    - Fixes bugs, errors, and faults after release.
    - Addresses problems reported by users or found in testing.
    - Does not add new features, only corrects existing issues.
    - Keeps the software working as originally intended.
    - **Example:** Fixing a crash issue in a mobile app.
  - **Adaptive maintenance:**
    - Modifies software to run in a new environment (OS, hardware, database, regulations).
    - Ensures compatibility with new platforms or technologies.
    - Responds to changes outside the system (not in the software itself).
    - Does not change functionality, only adjusts to the new setup.
    - Less urgent than the corrective maintenance
    - **Example:** Updating an app to run on Windows 11.
  - **Perfective maintenance:**
    - Modifications and updations to keep the software usable for longer period of time.
    - Adds new features or enhances existing ones.
    - Improves performance, speed, or usability.
    - Focuses on user satisfaction and efficiency.
    - **Example:** Adding dark mode to a mobile app.
  - **Preventive maintenance:**
    - Makes changes to prevent future faults or failures.
    - Comprises documentation updating, code optimization, and code restructuring
    - **Proactive:** done before problems occur.
    - **Example:** Cleaning old unused code to avoid errors later.

Fault Repair:

- **Changing a system to fix bugs/vulnerabilities** and correct deficiencies to meet its requirements
- Visualizes **corrective maintenance**; as it **focuses on correcting the system's faults.**
- **Coding errors are cheapest to fix**, **design errors cost more**, and **requirements errors are** the **most expensive** due to major redesigns.

Software Adaptation:

- Involves **modifying software to operate in a different environment** (hardware, OS, or platform) **than initially implemented**
- Visualizes **adaptive maintenance;** as it **focuses on adapting the system to new requirements or environmental changes**
- **Required when hardware, operating system, or support software changes.**
- **Ensures the system continues to function correctly in the new environment.**

Functionality Addition or Modification:

- **Making changes in software to add new features or improve existing ones**
- Helps to make the system **better, faster, and more efficient** without removing its original purpose
- Visualizes the concept of **perfective maintenance**; as it **focuses on improving the software by adding new features and enhancing its performance and structure**
- **Example:** Adding a new payment option in an app or improving its speed
- **According to surveys by Lientz and Swanson (1980) and Nosek and Palvia (1990), the maintenance effort is distributed as follows**:
  - **65% for implementing new requirements,**
  - **18% for adapting to new environments,** and
  - **17% for correcting system faults**

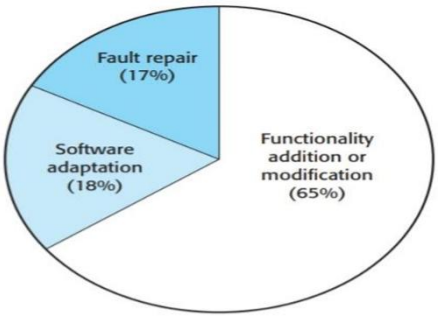


Fig: Maintenance effort distribution

Maintenance Prediction:

- Maintenance prediction means **estimating how much maintenance a software will need in the future**
- Studies **how easy the software is to maintain**, **what changes might be required**, **and how much it will cost**
- Helps to **predict which parts of the system will be difficult or costly to maintain** and what changes may be needed
- **3 main parts of maintenance prediction**:
    - **Predicting Maintainability:**
        - Finds out **which parts of the system are hard to maintain**
        - Answers: ***"What parts of the system will be most expensive or difficult to maintain?"***
    - **Predicting System Changes:**
        - Tries to **estimate what changes will happen in the system**
        - Answers: ***"How many change requests will come?"*** &
          "Which parts of the system will be affected by those changes?"
    - **Predicting Maintenance Costs:**
        - Estimates **how much money and effort will be needed for maintenance**
        - Answers: ***"What will be the cost of maintaining the system next year?"*** **&**
          "What will be the total (lifetime) maintenance cost?"

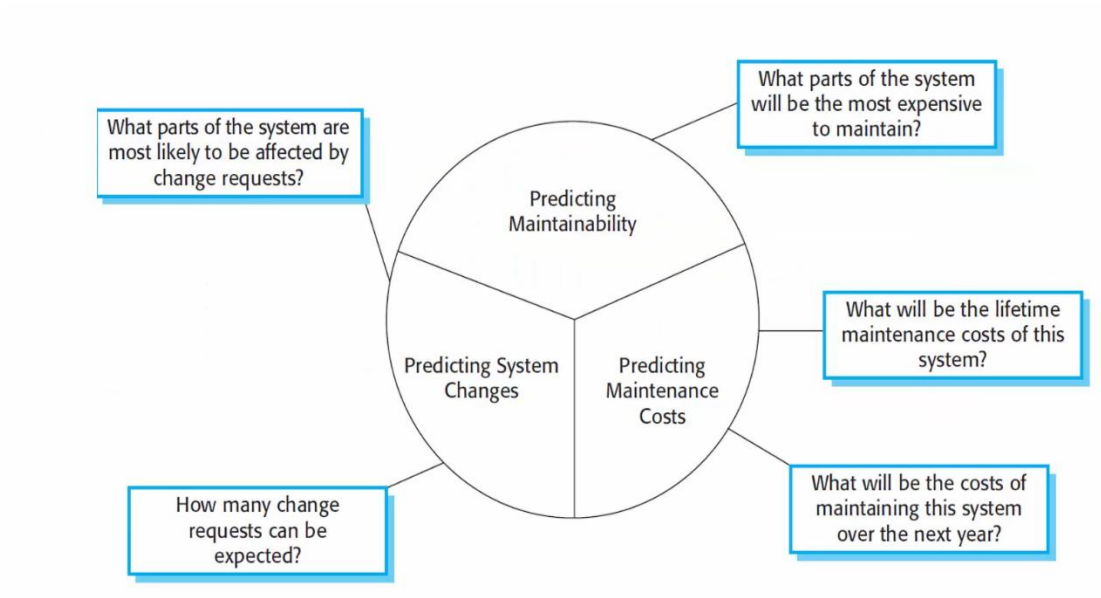


Fig: Maintenance Prediction

Software Re-Engineering:

- Software re-engineering means **analyzing and modifying existing software to improve its quality, performance, or maintainability without changing its basic functionality**
- Involves **reusing and improving old software instead of developing a new one from scratch.**
- **Advantages:**
  - **Saves cost and time** compared to building new software
  - **Improves maintainability and performance**
  - **Enhances software quality and reliability**
  - **Extends software life**
  - **Reduces risk**, since the system's basic logic is already proven
- **Software Re-Engineering Process:**
  - **Original program**: **Take the existing old software for improvement.**
  - **Source code translation**: **Convert the old program code to a modern programming language** or better format.
  - **Reverse engineering**: **Analyze the software to understand its design and functionality**, and create proper **program documentation**.
  - **Program structure improvement**: **Clean up the program to create** a re-structured, easier-to-maintain version, i.e., **restructured program**
  - **Program Modularization: Divide the restructured program into smaller, manageable modules** with the help of program documentation and **create the re-engineered program**
  - **Data Re-engineering: Clean up and restructure the original data** and create the re-engineered data

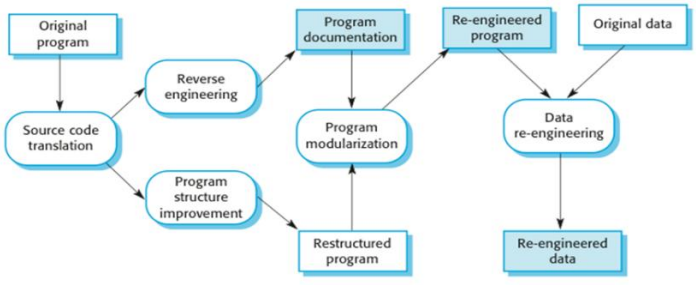


Fig: Re-Engineering Process

- **Re-engineering Cost Factors:**

i) Available reengineering tools **ii)** Availability of expert staff **iii)** Available reengineering tools

Reverse Engineering:

- Reverse engineering is a **process of analyzing software with a view to understanding its design and specification.**
- **May be a part of re-engineering process**, but sometimes it is useful on its own
- **Design and specification obtained through reverse engineering can serve as input for the requirements specification of a new/replacement system**
- **Reverse Engineering Process:**
  - **System to be re-engineered:** The existing software that needs improvement.
  - **Automated analysis / Manual annotation:**
    - **Automated analysis:** Tools automatically examine the system.
    - **Manual annotation:** Humans add notes or details where needed.
  - **System information store:** Stores all information from the analysis, including program and data structures, control flow, and dependencies.
  - **Document generation:** Using the stored information, documents and diagrams are created.
  - **Outputs/Diagrams:**
    - **Program structure diagrams:** Show program components and their relationships
    - **Data structure diagrams:** Show how data is organized and related within a system
    - **Traceability metrics:** Show relationships between requirements, code, and data.

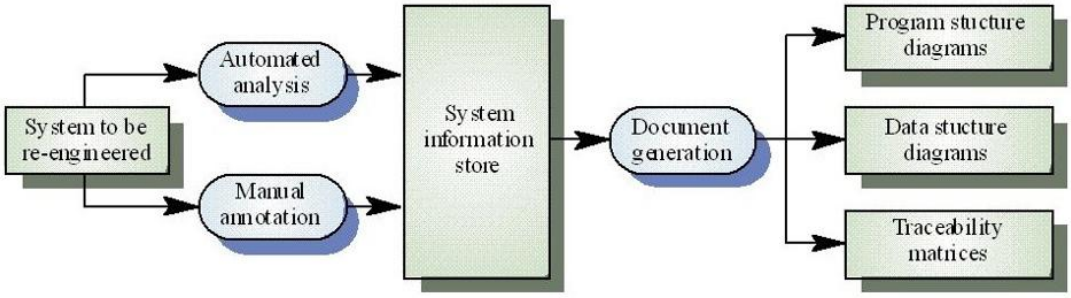


Fig: Reverse Engineering Process

Software Configuration Management (SCM):

- "SCM is the **process of identifying the items (components) in the system, controlling the change of the items throughout their life cycle, recording and reporting the status of items and change requests, and verifying the completeness and the correctness of items.**" [IEEE]
- We, **firstly, must identify all the configuration items (CI) for a software** and **then in the next step, we must define baseline for them i.e. original scope and design of a software product**
- This **baseline product undergoes continuous change** such that the **original quality and integrity of the product is always maintained.**
- **For SCM activities, the following documents are required:**
  - **Project plan**
  - **SRS (Software Requirement Specification) document**
  - **SDD (Software Design Description) document**
  - **Source code listing**
  - **Test plans / Test cases**
  - **User manuals**
- **SCM Process:**
  - **Identification of Software Configuration Items (SCIs):**
    - **Identifies the CIs (any item in the system whose change must be tracked and managed)** from products that compose baselines at given points in time
    - **Baseline is a set of mutually consistent configuration items, which has been formally reviewed and agreed upon,** and serves as the basis of further development
    - **Establishes relationships among CIs** and **defines control mechanisms and procedures for managing changes**
  - **Change Control:**
    - **Tracks and evaluates change requests (CRs)** submitted by customers or developers
    - **Assesses technical impact, side effects, dependencies, and cost of proposed changes**
    - **Ensures only approved changes are implemented** in the baseline
    - **Produces a change report summarizing analysis** and decision for each CR
  - **Version Control:**
    - **Manages and maintains multiple versions of configuration items**
    - **Ensures that changes made by different developers do not conflict with each other**
    - **Provides procedures and tools to record, retrieve, and manage different versions of CIs**

- **Configuration Auditing:**
  - **Formal technical review of the process and product**
  - **Verifies** that the **modified configuration items are complete, correct, and consistent**
  - **Confirms** that **all approved changes are properly implemented**
  - Acts as a **"watchdog" to preserve integrity of the system** and project scope
- **Status Reporting:**
  - **Tracks each version/release and the changes** that led to that version
  - **Maintains records of all modifications** from one baseline to the next
  - **Helps extract previous versions** for testing, validation, or maintenance

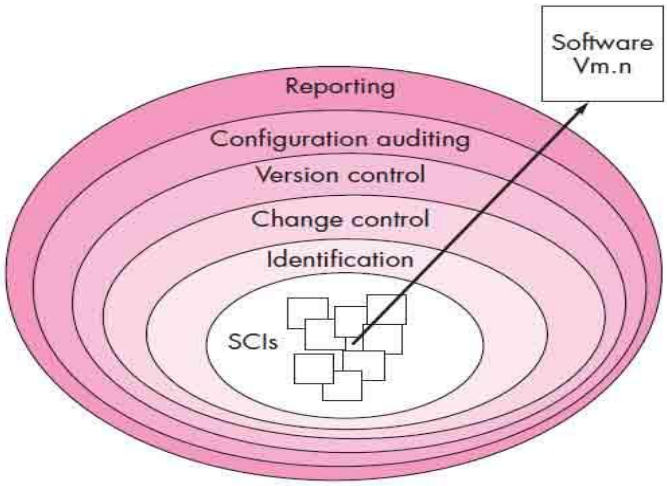


Fig: Layers of SCM Process

- **Importances of Configuration Management (CM):**
  - **Helps control and track changes** in software
  - **Improves quality by auditing and reviewing changes**
  - **Prevents conflicts when multiple people work on the same system**
  - **Control the costs involved in making changes** to a system
  - **Helps to develop the co-operation and co-ordination among the stakeholders**